

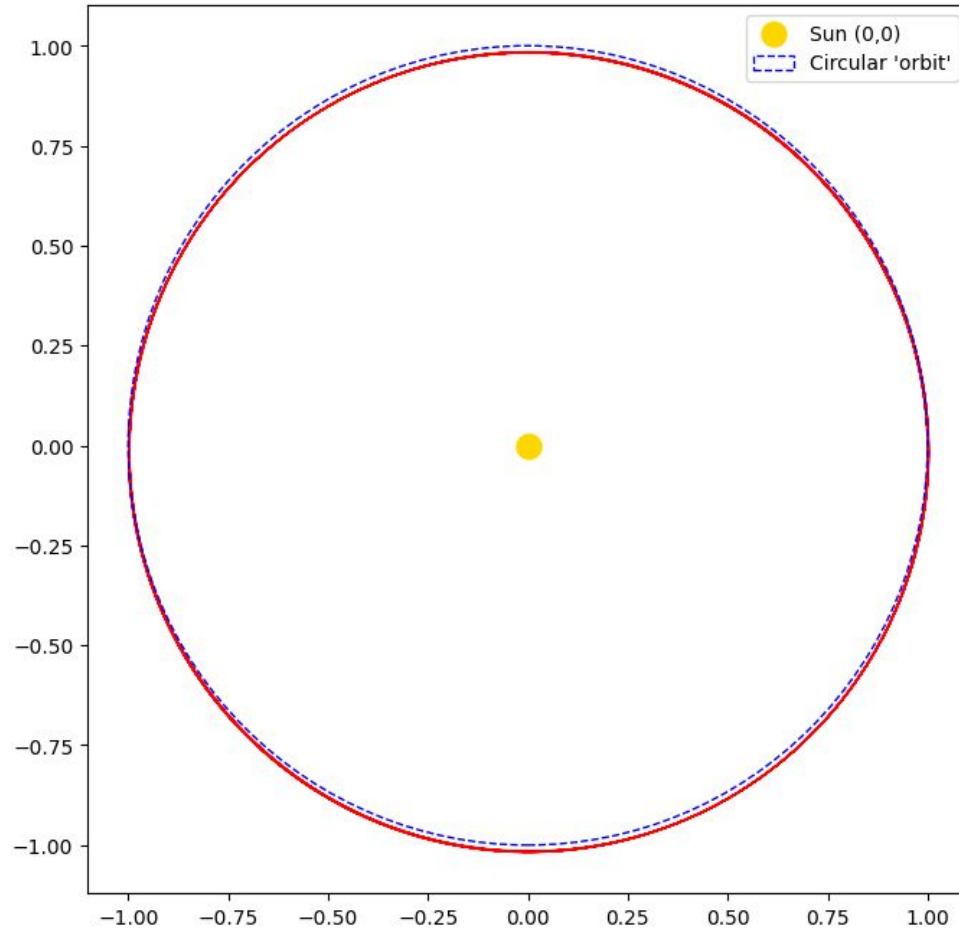
Kepler y Bayes se encuentran en un bar

Carlos J. Gil Bellosta
cgb@circiter.es



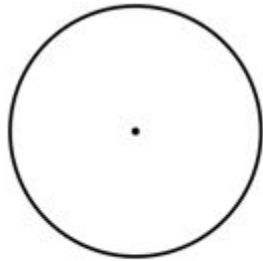
Kepler learned about these theories in his studies, and he made this observation that the

- Datos pequeños
- Persuasión
- *Tooling*

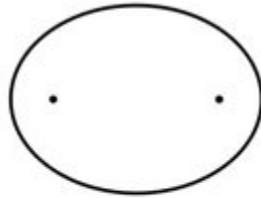


$$e \approx 0.0167$$

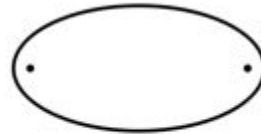
Excentricidad:



$$b/a=1$$
$$e=0$$



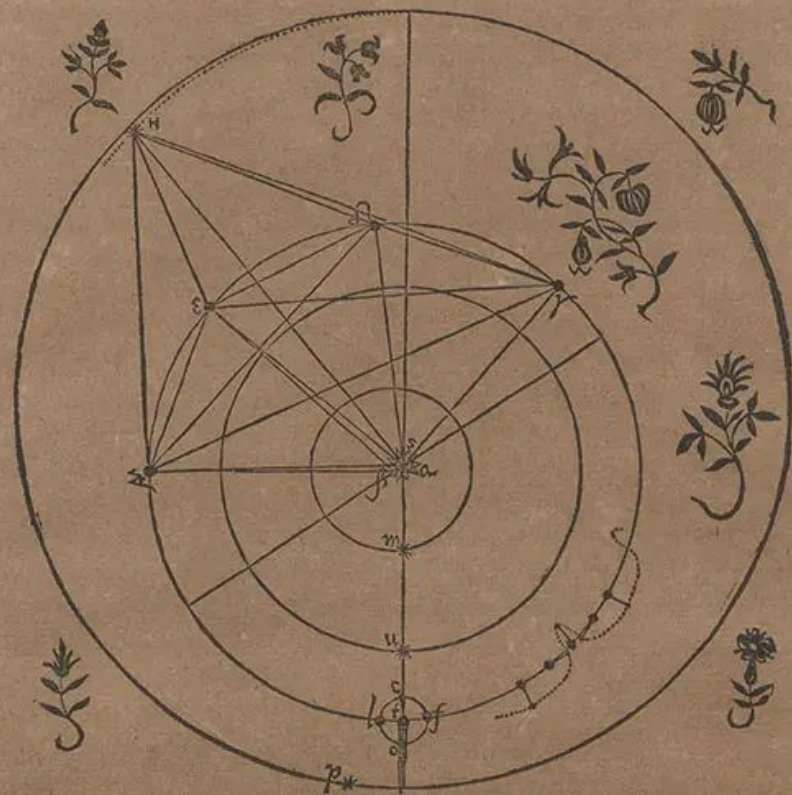
$$b/a=3/4$$
$$e=.6614$$



$$b/a=1/2$$
$$e=.8660$$



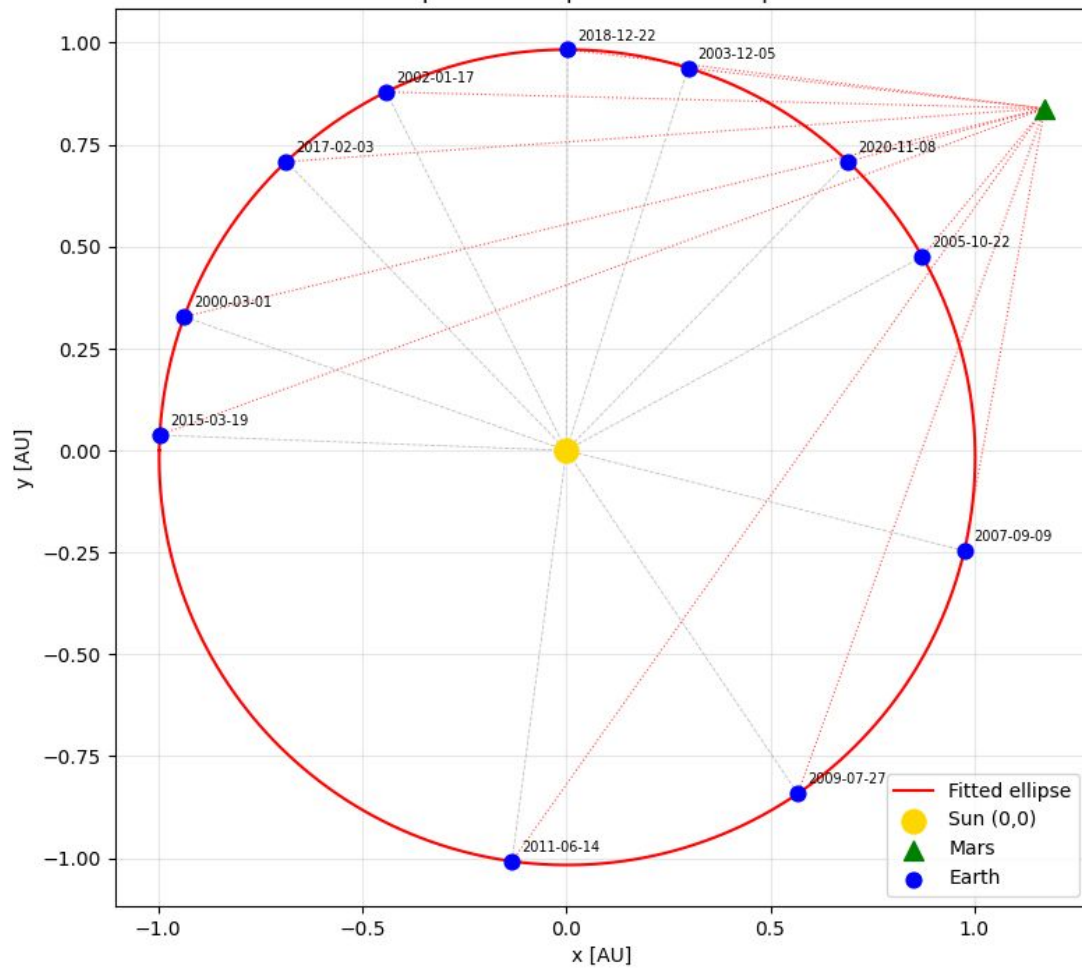
$$b/a=1/4$$
$$e=.9682$$



erunt anguli cogniti

Hinc dantur

Computed Earth positions and ellipse fit



Tycho Brahe's Mars Observations

The Orbit as Calculated with Modern Methods

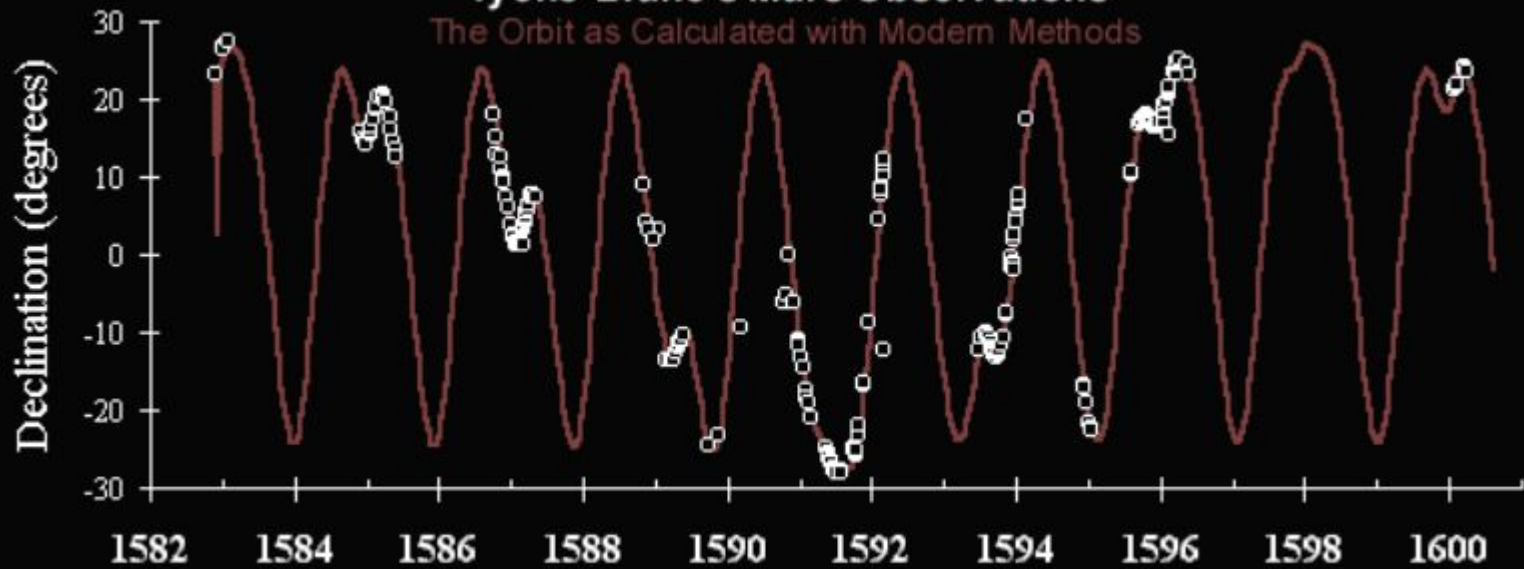
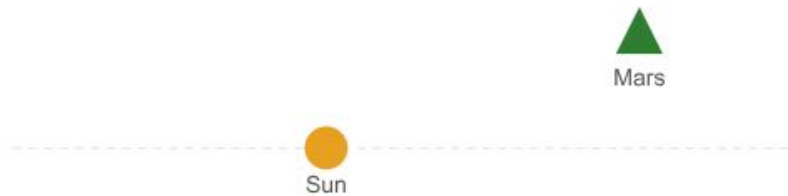
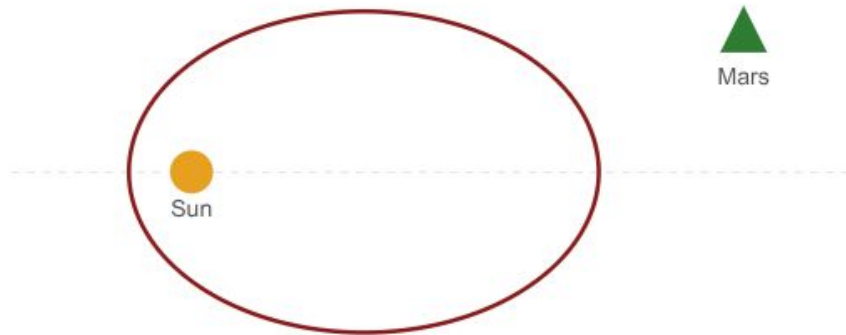


Image Copyright 2000, Wayne Pafko

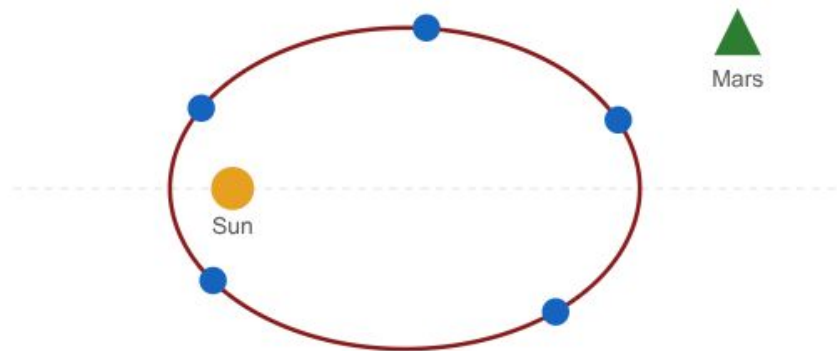
1



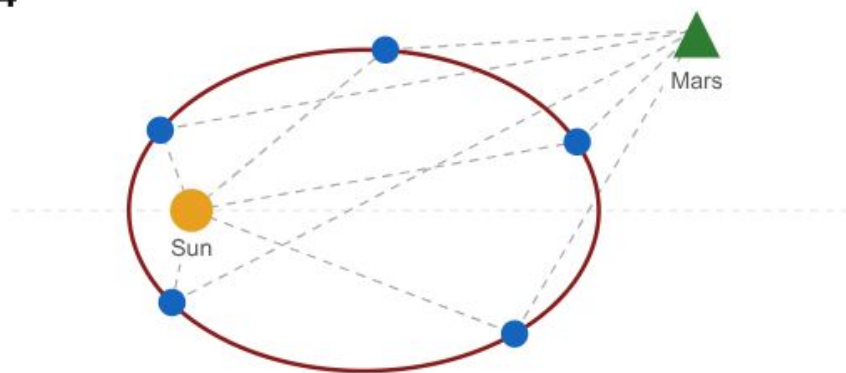
2



3



4



```

def objective(params):
    mars_x, mars_y, omega, e = params

    # Clamp e to valid range (avoid degenerate ellipses)
    if e <= 0 or e >= 1:
        return 1e10

    x, y = points_ellipse(selected_df['Sun Ecliptic Longitude (deg)'].values / 360 * 2 * np.pi, e, omega)
    return fitting_error(x, y, mars_x, mars_y,
                        selected_df['Mars Ecliptic Longitude (deg)'].values / 360 * 2 * np.pi)

x0 = [1.4, 0.3, np.pi, 0.01]

result = minimize(objective, x0, method='Nelder-Mead',
                  options={'xatol': 1e-8, 'fatol': 1e-8, 'maxiter': 50000})

mars_x, mars_y, omega, e = result.x
print(f"mars_x = {mars_x:.4f}")
print(f"mars_y = {mars_y:.4f}")
print(f"omega = {omega:.4f} rad ({np.degrees(omega):.2f}°)")
print(f"e = {e:.4f}")
print(f"loss = {result.fun:.6e}")
print(f"success: {result.success}")

mars_x_approx = mars_x
mars_y_approx = mars_y
omega_approx = omega

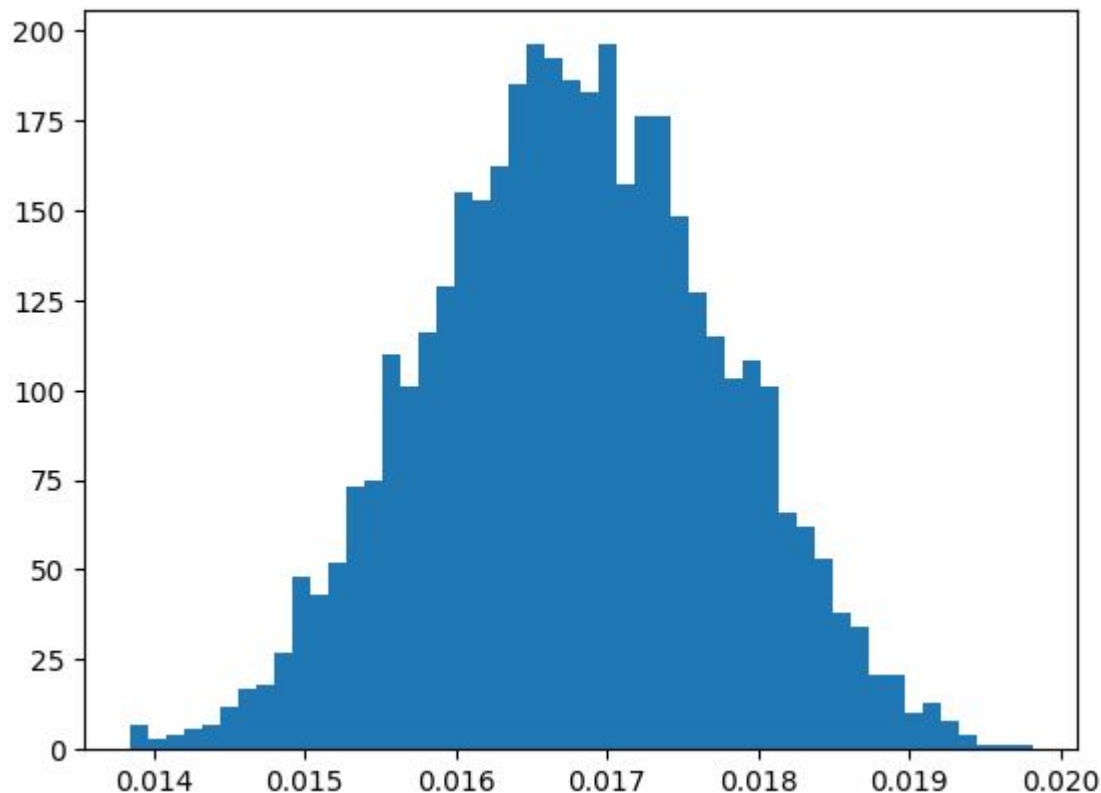
```

```

mars_x = 1.1724
mars_y = 0.8389
omega = 1.6872 rad (96.67°)
e = 0.0170
loss = 6.342178e-06
success: True

```

Distribución a posteriori de e



```

def model(sun_longitudes):

    # -----
    # pre-priors (to avoid sampling issues)
    # -----
    mars_x_error = numpyro.sample('mars_x_error', dist.Normal(0.0, 1.0))
    mars_y_error = numpyro.sample('mars_y_error', dist.Normal(0.0, 1.0))
    e_error      = numpyro.sample('e_error',      dist.HalfNormal(1.0))

    # -----
    # proper priors
    # -----

    mars_x = numpyro.deterministic('mars_x', mars_x_approx + mars_x_error * 0.01)
    mars_y = numpyro.deterministic('mars_y', mars_y_approx + mars_y_error * 0.01)
    omega  = numpyro.sample('omega', dist.Normal(omega_approx, 0.5))
    e       = numpyro.deterministic('e', e_error * 0.01)
    observations_sigma = 0.001

    # -----
    # actual model
    # -----

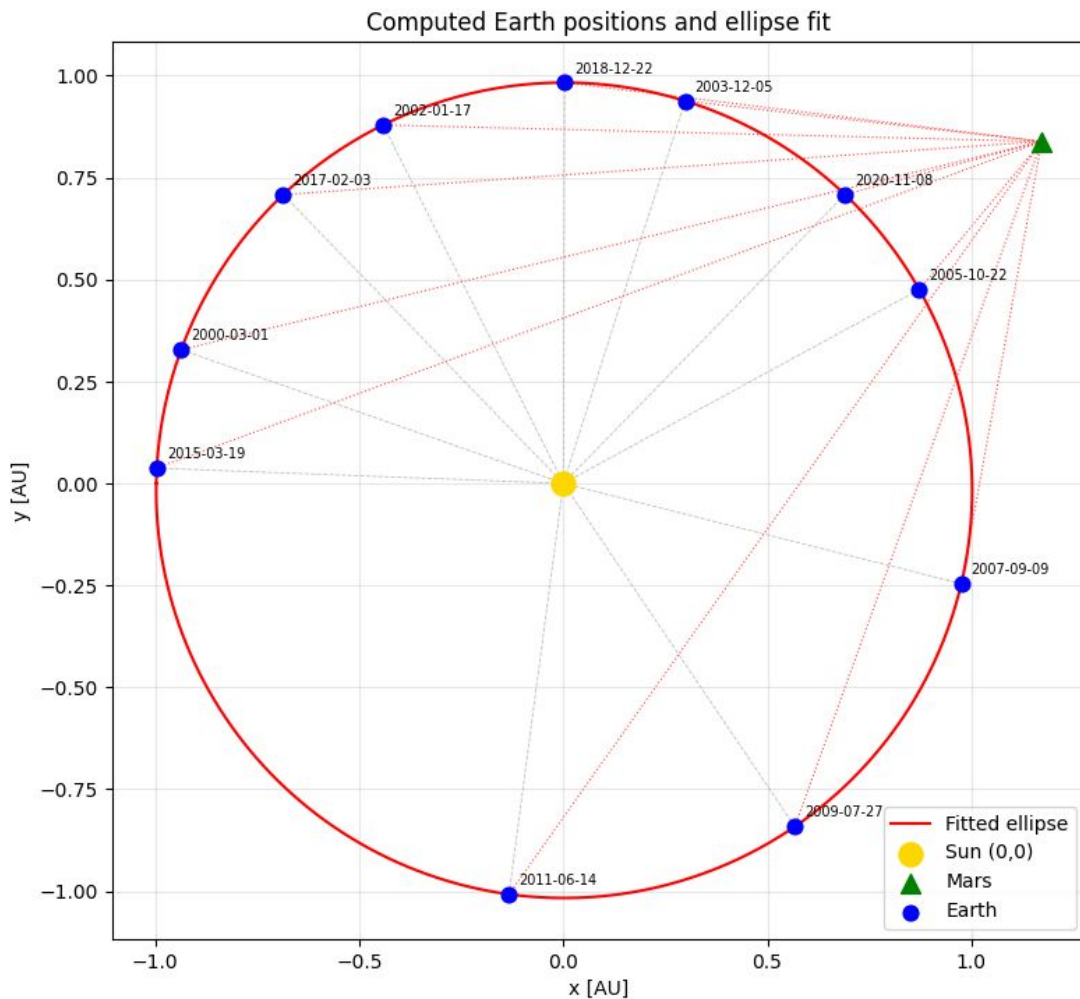
    # place the Earth on a "random ellipse"
    ex, ey = points_ellipse_alt(sun_longitudes, e, omega)

    # estimate Mars longitudes from the Earth positions
    tans = (mars_y - ey) / (mars_x - ex)
    mars_longitudes_mu = jnp.arctan(tans)

    # match estimated longitudes to observed ones
    numpyro.sample("mars_longitudes", dist.Normal(loc=mars_longitudes_mu, scale = observations_sigma))

    return(mars_longitudes_mu)

```



Carlos J. Gil Bellosta
cgb@circiter.es